# Práctico 09 — Iluminación, sombreado y luces

## Computación gráfica y ambientes virtuales (0494)

Facultad de Ingeniería
Centro Regional Universitario Córdoba - IUA
Universidad de la Defensa Nacional

30 de octubre de 2026






# Cómo modelar la luz sobre el terreno

**Antecedentes**: terreno con relieve y normales (Práctico 07), textura de pasto y cielo con un cubemap (Práctico 08). La textura reemplaza el color.

**Requisitos del proyecto**

- Calidad gráfica: iluminación.
- Al menos tres efectos

**Cómo completar requisitos**

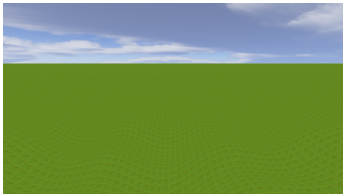
- Calcular, en cada fragmento, cuánta luz llega del sol y cuánta se refleja hacia el ojo (modelo de iluminación de Phong)
- Modificar el color en cada fragmento aplicando algún modelo especial (no es iluminación), ej. niebla

**Dos datos que no se usaron aún**:

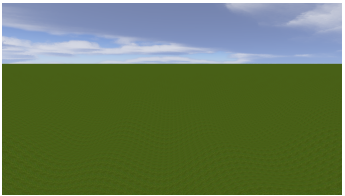
- La normal: que está en el `Vertex` desde el Modelado de primitivas y el terreno
- La textura (parcialmente): que reemplazó al color y que hoy pasa a multiplicarse por la luz.

# La misma geometría, tres imágenes

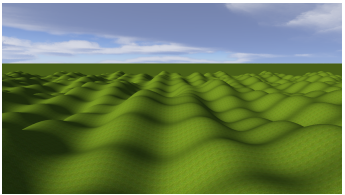
**Sin luz**: como el Práctico 08



**Con luz**: normal constante $(0, 1, 0)$



**Con luz**: normales del Práctico 07



Misma escena (terreno, cámara y textura), lo único que cambia es la **normal** que recibe el *fragment shader*.

El relieve está en la geometría desde el Práctico 07, pero no aparece en la imagen hasta que se aplica un modelo de iluminación. Con una normal constante la luz llega igual a todos los puntos y el terreno vuelve a verse plano, sólo más oscuro.

En este práctico se utilizan las normales que ya están calculadas (pero no verificadas).

# Un fragmento no es un píxel

Entre el *vertex shader* y el *fragment shader* está el *digitalizador* (rasterizador): por cada píxel que cubre un triángulo se genera un **fragmento**.

**Qué contiene un fragmento**

- Su posición en la imagen y profundidad en la escena.
- El valor de cada variable `out` del *vertex shader*, interpolado entre los tres vértices del triángulo.

**Por qué no es un píxel**

- Es un **candidato** porque el test de profundidad lo puede descartar.
- Varios triángulos generan fragmentos para el mismo píxel y sólo queda el más cercano.
- Con *multisampling* un píxel junta varias muestras.

Lo que el código no dice

Una variable `out vec3 fNormal` del *vertex shader* llega al *fragment shader* como `in vec3 fNormal` pero al pasar de una etapa a otra el rasterizador la **interpola** (con corrección de perspectiva). No hay una línea de código que lo pida: es el mecanismo por defecto. El calificador `flat` permite desactivar este mecanismo.

# La normal interpolada no es unitaria

Interpolar linealmente dos vectores unitarios da un vector de menor módulo. Sobre una arista entre $\bar{n}_1$ y $\bar{n}_2$ (unitarias, separadas un ángulo $\theta$):

$$\left|(1-t)\,\bar{n}_1 + t\,\bar{n}_2\right|^2 = 1 - 2t(1-t)(1-\cos\theta)$$

**Cuánto se achica la normal resultante**

- Vale 1 en los vértices; mínimo en $t = \frac{1}{2}$: $\left|\frac{1}{2}(\bar{n}_1 + \bar{n}_2)\right| = \cos\frac{\theta}{2}$.
- Para $\theta = 90°$: 0,707. Para $\theta = 30°$ (esfera de 6 anillos): 0,966.
- Dentro de un triángulo, lejos de los vértices, es donde más se achica.

**Por qué importa**

- La iluminación se calcula con productos escalares contra $\bar{n}$: si $\bar{n}$ mide 0,966, el producto sale un 3 % más chico.
- En el difuso, un 3 % no se nota.
- El brillo especular (próximo bloque) eleva ese producto a una potencia, típicamente 32: $0{,}966^{32} \approx 0{,}33$. El brillo cae a un tercio.

La regla: `normalize` en los dos shaders

*Vertex*: normales de módulo distinto sesgan la **dirección** interpolada.
*Fragment*, **siempre**: la interpolación no preserva la **longitud** (igual para $\bar{l}$ y $\bar{v}$ interpolados).

# Modelo de iluminación de Phong

El modelo de Phong con una luz puntual, como se dio el miércoles:

$$I = \underbrace{I_a\, k_a}_{\text{ambiente}} + \underbrace{I_p\, k_d\, (\bar{n} \bullet \bar{s})}_{\text{difusa}} + \underbrace{I_p\, k_s\, (\bar{v} \bullet \bar{r})^{n_s}}_{\text{especular}}$$

Esta ecuación hay que resolverla al momento de determinar el color del cada *pieza de superficie* y requiere conocer la posición ($P$) de la misma:

| teórico | código | qué es | de dónde sale |
|---|---|---|---|
| $\bar{n}$ | `N` | normal de la superficie | atributo del vértice, transformado e interpolado |
| $\bar{s}$ | `L` | dirección **hacia** la luz | `normalize(posLuz - P)` |
| $\bar{v}$ | `V` | dirección **hacia** el ojo | `normalize(posOjo - P)` |
| $\bar{r}$ | `R` | $\bar{s}$ reflejado en $\bar{n}$ | `reflect(-L, N)` |

Los cuatro vectores tienen que ser **unitarios** y estar en el **mismo sistema de coordenadas**. Usando el modelo RGB la ecuación se evalúa por canal: $I_p$ es el color de la luz y $k_d$ y $k_s$ rep. el color que emite el material (`vec3`).

# Iluminación Phong en GLSL

Implementación para una luz puntual. Todos los vectores en coordenadas de mundo:

```
vec3 phong(vec3 P, vec3 N)                          // N ya normalizada
{
    vec3 L = normalize(uLuzPos - P);                // hacia la luz
    vec3 V = normalize(uOjo - P);                   // hacia el ojo
    float NdotL = max(dot(N, L), 0.0);

    vec3 ambiente  = uAmbiente * uKd;               // ka = kd es lo habitual
    vec3 difusa    = uLuzColor * uKd * NdotL;
    vec3 especular = vec3(0.0);
    if (NdotL > 0.0) {
        vec3 R = reflect(-L, N);
        especular = uLuzColor * uKs * pow(max(dot(R, V), 0.0), uBrillo); // uBrillo = ns
    }
    return ambiente + difusa + especular;
}
```

- `max(..., 0.0)`: del lado opuesto a la luz el coseno es negativo y sin el limitador la luz se restaría.
- El `if` evita que aparezca brillo especular en la cara que no recibe luz.
- `reflect(I, N)` espera la dirección **incidente** de la luz *hacia* la superficie: por eso hay que negar L (`-L`).

# Un solo sistema de coordenadas para todos los vectores

**Coordenadas de mundo** (el ejemplo)

- $P$ y $N$ se pasan a mundo en el *vertex shader*.
- Las posiciones de la luz y el ojo llegan como `uniform` también en mundo.

**Coordenadas de ojo** (Shader Cookbook - Wolff)

- El ojo está en el origen: $V =$ `normalize(-P)`.
- La posición de la luz hay que pasarla a la referencia ojo en cada cuadro.

```
// Codigo en vertex shader
vec4 pm      = uModelo * vec4(vPos, 1.0);
fPos         = pm.xyz;                          // P, en mundo
fNormal      = uMatNormal * vNormal;            // N, en mundo (transformación a cont.)
gl_Position  = uVistaProy * pm;
```

La posición del ojo en mundo puede salir de la matriz de vista o pasarse como `uniform` desde el CPU:

```
// Codigo en vertex shader
glm::vec3 ojo = glm::vec3(glm::inverse(vista)[3]);    // la cuarta columna
```

Mezclar los dos sistemas no da error: el síntoma es que la luz se mueve con la cámara.

# Como transformar las normales: la matriz normal

Una normal **no se transforma** con la matriz de modelo. Se tiene que conservar la perpendicularidad con la superficie:

Una tangente $\bar{t}$ sí se transforma con el modelo (es la diferencia entre dos puntos): $\bar{t}' = M\,\bar{t}$. Se busca una matriz $G$ que lleve $\bar{n}$ a $\bar{n}' = G\,\bar{n}$ con $\bar{n}' \bullet \bar{t}' = 0$:

$$\bar{n}^T G^T M\,\bar{t} = 0 \text{ si } G^T M = I \;\Rightarrow\; G = \left(M^{-1}\right)^T$$

Una normal es una dirección y la traslación no la afecta $\rightarrow$ de $M$ se remueve la traslación y queda de $3 \times 3$.

**Transf. que NO afectan la normal**

- Rotaciones (y reflexiones): $\left(M^{-1}\right)^T = M$.
- Escala uniforme: la misma dirección con otra longitud.
- Traslaciones: no afectan a las normales

**Transf. que SI afectan la normal**

- Escala no uniforme
- Cizallas

Se calcula de lado del CPU, **una vez por objeto** y se pasa como `uniform`:

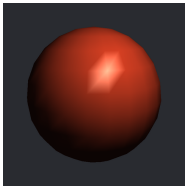
```
glm::mat3 matNormal = glm::transpose(glm::inverse(glm::mat3(modelo)));
glProgramUniformMatrix3fv(prog, locMatNormal, 1, GL_FALSE, glm::value_ptr(matNormal));
```

# Gouraud y Phong: en qué *shader* se evalúa la ecuación

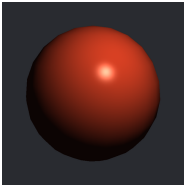
**Gouraud**: por vértice

```
// vertex shader
vec3 N = normalize(uMatNormal * vNormal);
fColor = phong(pm.xyz, N);
// fragment shader
fragColor = vec4(fColor, 1.0);
```

Se interpola el **color**.



Gouraud



Phong

**Phong**: por fragmento

```
// vertex shader
fPos    = pm.xyz;
fNormal = uMatNormal * vNormal;
// fragment shader
vec3 N = normalize(fNormal);
fragColor = vec4(phong(fPos, N), 1.0);
```

Se interpola la **normal**.

La función `phong()` es **la misma**; cambia dónde se llama. Con Gouraud el brillo especular sólo existe si cae sobre un vértice.

"Phong" se refiere a dos modelos: el **modelo de iluminación** (la ecuación) y el **modelo de sombreado** (interpolar la normal).

# Sombreado plano: `flat` y su trampa

**El calificador** `flat`: indica que no hay que interpolar

```
flat out vec3 fNormalPlana;
```

Sin interpolación, todos los fragmentos del triángulo reciben el valor de un mismo vértice, el **vértice provocador** (por omisión, el tercero del triángulo). La trampa: en una malla indexada ese vértice es compartido y tiene la normal **suavizada**, no la de la cara. Las facetas salen **en diagonal**.



`flat`: facetas en diagonal

**La normal de la cara calculada con derivadas**

```
vec3 N = normalize(cross(dFdx(fPos), dFdy(fPos)));
```

La GPU procesa los fragmentos en bloques de $2 \times 2$: `dFdx(fPos)` es la diferencia con el vecino en $x$, y `dFdy` con el vecino en $y$. Los dos están en el plano del triángulo $\rightarrow$ su producto vectorial es la normal de la cara.



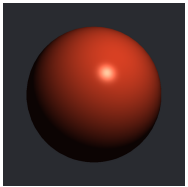
derivadas: la cara

# Sombreado mejorado Blinn-Phong: el vector medio

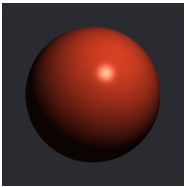
En el *pipeline* fijo se evitaba reflejar $L$ en cada vértice: con luz y ojo lejanos, $H$ es constante para toda la escena. Se usa un vector medio ("a mitad de camino") entre la luz y el ojo como aproximación:

$$H = \frac{L+V}{|L+V|} \qquad I_{spc} = I_p\, k_s\, (N \bullet H)^{n'} \qquad n' \approx 4\, n_s$$

El vector $H$ representa la dirección de máxima especularidad para una posición fija de la fuente de luz.



Phong, $n_s = 32$



Blinn-Phong, $n' = 128$

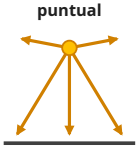
**Por qué se usa**

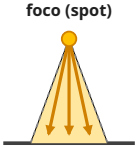
- $R \bullet V$ se anula cuando el ángulo entre $R$ y $V$ pasa de 90°, aunque la cara esté iluminada y a la vista: con exponentes bajos el brillo termina en un **borde marcado**. $N \bullet H$ es positivo mientras la luz y el ojo estén del lado visible.
- Es el que usaba el pipeline fijo de OpenGL (`glLight`, legado) y el que trae casi todo el código que se encuentra.

Con el exponente multiplicado por 4 se obtienen resultados muy similares que casi no se distinguen.

# Modelos de luces: 3 tipos




## Direccional

- Fuente en el infinito
- Dirección $L$: igual en todas las sup.
- Ej.: sol en una escena

## Puntual

- Fuente en una posición finita
- Dirección $L$ cambia en cada sup.
- Se puede atenuar con la distancia ($f_{att}$)

## Foco (*spot*)

- Puntual con volumen de cobertura definido
- Fuera del volumen (cono) no ilumina
- Ej. Linterna, luces de aterrizaje

Ejemplo modelado para luces Direccional y Puntual:

Un vec4 como en homogéneas: $(x, y, z, 1)$ es un punto y $(x, y, z, 0)$ el punto en el infinito en esa dirección (luz direccional).

```
vec3 L = (uLuz.w == 0.0) ? normalize(uLuz.xyz)          // direccional: sin posicion
                         : normalize(uLuz.xyz - P);     // puntual: usa la posicion
```

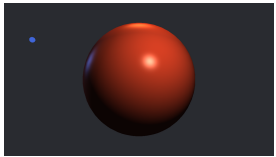
# Varias luces

La luz es aditiva: se evalúan las componente difusa y especular de cada fuente y se suman. La componente ambiental se suma una sola vez.

```
// Ejemplo componente difusa
layout (location = 8)  uniform int  uNumLuces;
layout (location = 20) uniform vec4 uLuzPos[3]; // 20 a 22
layout (location = 23) uniform vec3 uLuzColor[3];

vec3 luz = uAmbiente;
for (int i = 0; i < uNumLuces; ++i) {
    vec3 L = ...;   // como en la filmina anterior
    luz += uLuzColor[i] * max(dot(N, L), 0.0);
}
```

```
glProgramUniform4fv(prog, 20, 3, glm::value_ptr(pos[0]));
```



Tres luces: principal (cálida), relleno (azulado, aclara la sombra) y contraluz (marca el borde)

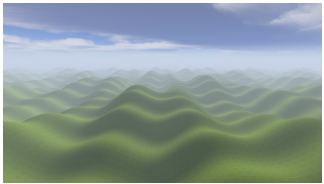
- La suma de contribuciones puede pasar de 1 y el color se recorta: aparecen zonas **quemadas** en blanco. Por eso las intensidades se eligen para que sumen cerca de 1 (p. ej., principal 0,7 y relleno 0,3).

# Iluminar una textura

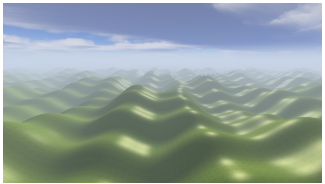
En el Práctico 08 la textura fijaba el color de la superficie. Ahora da los coeficientes ambiental y difuso del material del modelo de Phong ($k_a = k_d$, uno por texel), que multiplican a la luz.

```
vec3 kd    = texture(uPasto, fUV).rgb;                  // antes: fragColor = texture(...)
vec3 color = kd * (ambiente + difusa) + ks * especular;
```

La componente especular no se aplica a la textura: en un material no metálico (plástico, piedra, pasto) el brillo tiene el color de la luz.



Terreno sin comp. especular



Mismo terreno, con especular

$k_s$ y $n_s$ son por material: con valores altos toda la superficie brilla igual y se ve *húmeda*, de plástico.

# Efecto Niebla

Efecto sencillo que permite aumentar la sensación de profundidad en escenas chicas. Muy usado en video-juegos para dibujar pocos polígonos.

**Estrategia**: el color iluminado se mezcla con un color fijo (niebla) según la distancia $d$ al ojo. Tres formas habituales del factor $f$ de mezcla:

| | | |
|---|---|---|
| lineal | $f = \frac{d - d_0}{d_1 - d_0}$, recortado a $[0, 1]$ | nada hasta $d_0$, total desde $d_1$ (Wolff) |
| exponencial | $f = 1 - e^{-d/V}$ | (demos) |
| exponencial cuadrática | $f = 1 - e^{-(d/V)^2}$ | más limpia cerca, más cerrada lejos |

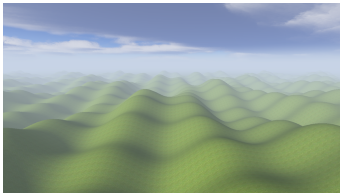
$V$ es la **visibilidad**: distancia (se elije) donde el color de niebla es el 63 %; al doble, el 86 %; al triple, el 95 %.

```
float f = 1.0 - exp(-length(fPos - uOjo) / uVisibilidad);   // d en mundo, como la luz
color = mix(color, uNiebla, f);
```

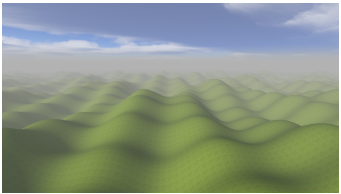
Se aplica **después** de la luz: la niebla es un color plano que no se ilumina.

# El color de la niebla es el del horizonte

Niebla del color del horizonte del cielo



Niebla gris fija



Con un gris fijo se forma una franja sobre el horizonte: el terreno lejano se funde con el gris pero el cielo no. Para un efecto más natural, el color se toma del cubemap: el promedio de la fila del horizonte de las cuatro caras laterales, calculado en la CPU al cargarlo.

## Mejora sobre el terreno del Práctico 08

Al terreno continuado del fondo hay que aplicarle la misma luz y la misma niebla que el terreno (la misma función en los dos *shaders*), si no aparece una discontinuidad en el borde de la ventana de terreno. El cielo se mezcla con la niebla sólo cerca del horizonte: a elevación cero, cielo y terreno valen el color de la niebla.

# Mejorar la apariencia sin modificar la geometría

Para que una superficie se vea rugosa (piedras, juntas, óxido) sin modelar cada relieve con polígonos, el detalle se guarda en una textura. Se puede usar para mover la geometría o sólo para engañar a la luz:

| | **desplazamiento** | **bump mapping** | **normal mapping** |
|---|---|---|---|
| la textura guarda | alturas | alturas | la normal |
| qué modifica | la **posición** de los vértices | la normal, desde las derivadas de la altura | la normal, se lee directo |
| la silueta | cambia | no cambia | no cambia |
| costo | muchos vértices | varias lecturas por fragmento | una lectura |



polyhaven: desplazamiento



la demo: normal map

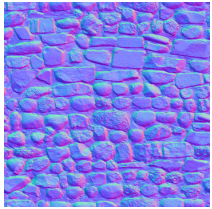
Las mismas texturas. En el render de polyhaven las piedras **sobresalen del contorno**; en la demo el contorno es el de la esfera, liso. El normal map engaña a la luz, no a la silueta.

polyhaven provee los dos mapas: *Displacement* (alturas, en gris) y *Normal*.

# Qué guarda un normal map

## Cada texel es una dirección

- Idea: codificar la normal local de cada fragmento como un color y guardarlo en una imagen
- Cada texel guarda la normal de un punto de la superficie. El color va de 0 a 1 y la normal de $-1$ a 1: se recupera con $\bar{n} = 2\,\mathrm{rgb} - 1$.
- Las componentes están en el **espacio tangente**: $x$ a lo largo de $u$, $y$ a lo largo de $v$, $z$ hacia afuera de la superficie.
- Para llevarla a mundo hay que saber hacia dónde corren $u$ y $v$ de la textura sobre la superficie ($T$ y $B$). La demo lo calcula en el *fragment shader* con `dFdx` y `dFdy`; el detalle está en el enunciado.
- Sin relieve la normal es $(0, 0, 1)$, o sea rgb $= (0{,}5;\ 0{,}5;\ 1)$: por eso los normal maps son celestes.



`rock_wall_17_nor_gl_1k.png`

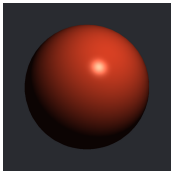
## Es un dato, no un color

- No se trata como sRGB (vuelve con gamma).
- polyhaven lo da en PNG de 16 bits: `stbi_load_16` y `GL_RGB16`.

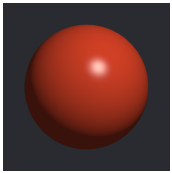
# Corrección gamma: la luz se suma en lineal

Lo que se vio en la Unidad III: la pantalla no es lineal, y las imágenes y los colores elegidos a ojo vienen codificados en **sRGB** ($\gamma \approx 2{,}2$). La ecuación de Phong suma y multiplica intensidades: tiene que operar sobre valores **lineales**.

```
vec3 kd = pow(texture(uDifusa, fUV).rgb, vec3(2.2));     // sRGB -> lineal
vec3 color = kd * difusa + ks * especular;               // las cuentas, en lineal
fragColor = vec4(pow(color, vec3(1.0 / 2.2)), 1.0);      // lineal -> sRGB
```



sin corrección



con corrección

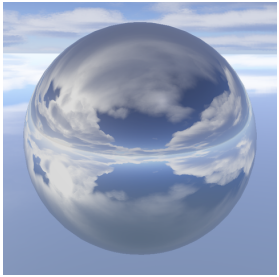
- El paso de la luz a la sombra se vuelve **más suave**, y el lado oscuro deja de ser negro.
- **Sólo el color** se convierte: el normal map y el mapa de alturas son datos.
- El hardware lo puede hacer solo: textura `GL_SRGB8_ALPHA8` y `glEnable(GL_FRAMEBUFFER_SRGB)`.

# Reflexión con el cubemap del cielo

Con la dirección en la que el ojo ve reflejado se lee el mismo `samplerCube` del cielo del Práctico 08:

```
vec3 R = reflect(-V, N);     // la visual, reflejada
vec3 cielo = texture(uCielo, R).rgb;
color = mix(color, cielo, 0.85);
```

- La esfera refleja el cielo, pero **no** el terreno ni a sí misma: el cubemap es lo único que conoce.
- Reflejar la escena de verdad es seguir el rayo reflejado: es el **raytracing** del 04-nov, con el mismo `reflect()`.



# El trabajo de la semana

## Práctico 09 — lo que hay que hacer

- **Iluminar el terreno** con el sol (luz direccional), usando las normales del Práctico 07.
- **La textura como $k_d$**: alguna que se elija, multiplicada por la luz.
- **Iluminar el avión**, con la matriz normal: las piezas escaladas son las que muestran si está bien.
- **Niebla** del color del horizonte del cielo, en el terreno y en el suelo continuado.
- Opcional: un **normal map** sobre alguna pieza.

## Lo que queda abierto

- **Sombras**: el modelo de Phong es **local**, cada punto se ilumina sin saber de los demás. El avión no proyecta sombra sobre el terreno.
- **Iluminación global y raytracing**: próximas clases. El reflejo del cubemap es lo más cerca que se llega sin salir del modelo local.
- **Cámaras y HUD**: texto y transparencia, después del parcial.
- La **presentación de avance** del proyecto es el **27-nov**: deseableq que el terreno esté iluminado.